Lesson 5

Tree-Based Algorithms & Ensembles

"The whole is something besides the parts."

— Aristotle, *Metaphysics*, Book VIII

Introduction & Motivation

The algorithms in Part I favored smooth geometry: linear planes, margins, kernels, and orthogonal projections. Many engineering systems are not smooth. They switch, saturate, trip, foul, overheat, crack, precipitate, or cross a phase boundary. A small change in flow rate, load, temperature, pressure, or composition can move the system into a different physical regime.

Tree-based models are designed for this kind of reality. Rather than forcing one smooth equation through the entire domain, they partition the state space into locally simple regions. This gives us three advantages at once: the model is easy to visualize, the first few splits provide a natural hierarchy of important features, and the final prediction path can be read as an engineering explanation. Ensembles, especially **Random Forests**, then stabilize these trees and turn a simple interpretable idea into one of the strongest practical algorithms for tabular engineering data.

Lesson Objectives:

- Explain why trees are useful for regime-switching physical systems.
- Distinguish learned decision trees from manually specified rule-based expert systems.
- Visualize the training process: candidate splits, impurity reduction, recursion, and stopping.
- Analyze how tree structure supports interpretability through hierarchical feature splits.
- Define maximum tree depth as a structural hyperparameter controlling the bias–variance tradeoff.

- Explain Random Forests as variance-reducing ensembles for robust tabular engineering prediction.
- Contextualize boosted-tree methods, including XGBoost, as widely used extensions of the basic tree framework.

Q&A: Decision Trees versus Rule-Based Expert Systems
Question:
If a trained decision tree can be written as a sequence of threshold rules, in what sense is it a learned statistical model?
Answer:
In a rule-based expert system, the variables, thresholds, and conclusions are prescribed a priori by a human expert. In **Classification and Regression Trees (CART)**, these quantities are estimated from data. The training algorithm selects the splitting variable, the numerical threshold, and the terminal-node prediction by optimizing a loss or impurity-reduction criterion. Thus, the learned split thresholds and leaf predictions play the role of model parameters, analogous to regression coefficients in a linear model.

5.1 The Limits of Continuous Models

In Part I, we utilized Ordinary Least Squares, Logistic Regression, and Support Vector Machines. These algorithms are mathematically robust, but they often prefer smooth boundaries or smooth response surfaces.

Many physical systems are instead governed by thresholds. A pump may remain stable until flow becomes too low. A motor winding may remain safe until cooling fails. A chemical product may remain dissolved until supersaturation is reached. A material may remain insulating until conductive pathways percolate. A structure may appear healthy until a crack reaches a critical length.

For readers outside chemical engineering, the key idea is simple: below a critical condition, the system is in one regime; above it, the system behaves qualitatively differently. A global smooth model often smears this transition. A tree can place a sharp split directly at the regime boundary.

Continuous linear models, and even very flexible smooth models, can struggle with these sudden changes unless they use many parameters.

Tree-based algorithms attack the problem differently: they partition the state space into engineering regimes.

5.2 Why Decision Trees Are Useful

A single decision tree has several practical advantages that are especially valuable in engineering courses and industrial discussions:

- **Visual explanation:** the trained model can be drawn as a tree of threshold tests.
- **Feature hierarchy:** the root split is usually the most important question the model learned. Later splits provide a natural ranking of secondary effects.
- **Mixed variables:** trees can handle nonlinear thresholds, interactions, and variables with different units without requiring feature scaling.
- **Regression and classification:** the same training logic predicts either continuous numbers or discrete labels.
- **Strong ensemble base learner:** a single tree is unstable, but many decorrelated trees form a Random Forest, which is often a reliable baseline for tabular data.

The Physical Reality: Interpretable Feature Hierarchies

The ordering of splits in a trained tree provides an interpretable hierarchy of variables. If coolant flow appears at the root, followed by heat load and ambient temperature, the model has identified a structured sequence of predictive tests. This hierarchy should not be interpreted as a formal causal proof, but it is a useful diagnostic summary of which variables dominate the prediction and which variables refine it.

5.3 CART: Classification and Regression Trees

The foundational building block of tree-based learning is the **CART** algorithm. The name reminds us that the same tree structure can solve both *classification* problems, such as safe versus unsafe operation, and *regression* problems, such as predicting a temperature or a yield.

Before writing the equations, it is useful to define the language of a tree. A **node** is a subset of the training data that has reached a particular

point in the tree. The first node is the **root node**; it contains all training observations. When the model asks a threshold question, for example "is the coolant flow below 2.8 L/min?", the node is divided into two **child nodes**. One child receives the observations for which the answer is yes, and the other receives the observations for which the answer is no. A node that is no longer split is a **leaf**. The leaf stores the final prediction.

For example, suppose a motor-pump dataset contains 40 experiments. Each experiment has measured inputs such as coolant flow, heat load, ambient temperature, and vibration. We can write one experiment as

$$x_i = [q_i,\ P_i,\ T_{\text{amb},i},\ v_i].$$

The quantity we want to predict is the **response** or **label**. For a regression tree, this may be the measured maximum casing temperature,

$$y_i = T_{\text{max},i}.$$

For a classification tree, it may be an overheating label, for example $y_i = 1$ for overheating and $y_i = 0$ for normal operation. Thus, y_i is not a mysterious mathematical object: it is the measured outcome attached to experiment i.

At the root node, CART may test the split $q < 2.8$ L/min. If 15 experiments have low coolant flow, they go to the left child node. The remaining 25 experiments go to the right child node. If the low-flow child is then split by heat load, for example $P > 70$ W, the tree has created smaller and more homogeneous engineering regimes. This is the entire idea: ask simple threshold questions until each region is easy to predict.

Mathematically, a decision tree partitions the input state space $\mathcal{X}$ into M disjoint regions $R_1, R_2, \ldots, R_M$. In the motor-pump example, one region might mean "low coolant flow and high heat load." Within each region, the model predicts one constant value c_m, such as the average measured T_{max} of the experiments that reached that leaf. The predictive function is:

$$f(x) = \sum_{m=1}^{M} c_m \mathbf{I}(x \in R_m) \tag{5.1}$$

where $\mathbf{I}$ is the indicator function. It equals 1 if the input x belongs to region R_m, and 0 otherwise. In words: the tree first decides which leaf the observation reaches, and then returns the value stored in that leaf.

Finding the globally optimal partition of a high-dimensional space is computationally difficult. CART therefore uses a **greedy, top-down recursive partitioning** approach. "Greedy" means that at each current node the algorithm chooses the best split available at that moment, rather than searching all possible future trees. For example, if the current node contains mixed safe and overheating experiments, CART tests candidate questions such as $q < 2.8$, $P > 70$, or $T_{\text{amb}} > 32$, and keeps the question that makes the two child nodes easier to predict.

Definition: Impurity Reduction at a Candidate Split

Assume the current node is called R. A candidate split divides it into a left child R_L and a right child R_R. CART asks: did this split make the children cleaner than the parent?

$$\Delta I = I(R) - \frac{|R_L|}{|R|} I(R_L) - \frac{|R_R|}{|R|} I(R_R). \tag{5.2}$$

Here $I(R)$ measures how mixed or uncertain the current node is. The two weighted terms subtract the remaining impurity in the children. If ΔI is large, the split was useful. For regression, $I(R)$ is usually mean squared error. For classification, $I(R)$ can be entropy or Gini impurity. The split with the largest ΔI is selected.

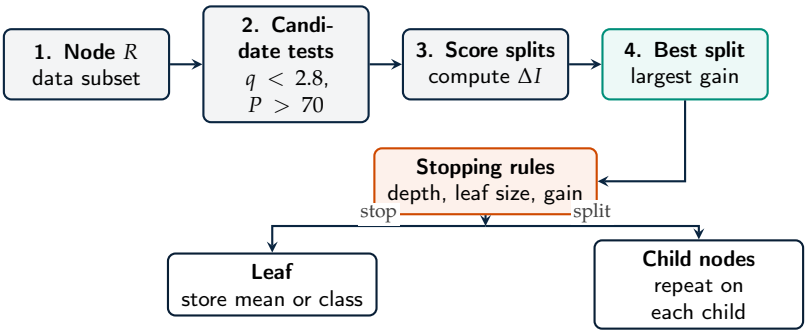


Figure 5.1: **CART training workflow.** At a node R, CART tests simple threshold questions, scores each candidate by impurity reduction, keeps the best split, and then applies stopping rules. If the node is split, the same procedure is repeated separately on each child node.

5.3.1 Splitting Criteria for Regression

In regression, each leaf predicts a number. For example, a leaf may predict that the maximum casing temperature will be 82°C. The training question is therefore: which split makes the numerical predictions inside the two child nodes as consistent as possible?

Consider a current node containing 20 experiments with different values of coolant flow q, heat load P, and measured maximum temperature $T_{\max}$. CART may test the candidate split $q < 2.8$ L/min. This creates two child nodes:

- the left child contains experiments with $q < 2.8$ L/min;
- the right child contains experiments with $q \geq 2.8$ L/min.

If the temperatures inside each child are tightly clustered around their local averages, the split is useful. If both children still contain a confusing mixture of low and high temperatures, the split is weak.

For this regression example, the split is scored by how tightly each child node clusters around its own average maximum temperature. The standard score is the **Sum of Squared Errors (SSE)**: add up the squared distance between each measured temperature and the average temperature of the child node that contains it. A small SSE means that each child node contains experiments with similar temperatures; a large SSE means that

the split still mixes very different outcomes. If feature j is split at threshold s, creating a left child with average temperature $\bar{T}_{\max,L}$ and a right child with average temperature $\bar{T}_{\max,R}$, the SSE is:

$$\mathrm{SSE}(j,s) = \sum_{i \in R_L(j,s)} \left(T_{\max,i} - \bar{T}_{\max,L}\right)^2 + \sum_{i \in R_R(j,s)} \left(T_{\max,i} - \bar{T}_{\max,R}\right)^2. \quad (5.3)$$

CART tries many possible choices of feature j and threshold s, and keeps the split with the smallest $\mathrm{SSE}(j,s)$. In compact machine-learning notation, $T_{\max,i}$ is often written as y_i, because y_i denotes the measured response we want to predict. The meaning here is concrete: for this example, y_i is the measured maximum temperature of experiment i.

For example, if the low-flow child has measured temperatures near 95°C, that child predicts a high temperature. If the high-flow child has temperatures near 62°C, that child predicts a lower temperature. The tree continues splitting only while the new split produces a meaningful improvement and the stopping rules still allow it.

5.3.2 Splitting Criteria for Classification

In classification, each leaf predicts a class or a class probability. For example, the target may be "normal operation" versus "overheating alarm." In this case, averaging numerical temperatures is not the right objective. Instead, the tree tries to create child nodes that are **pure**: each child should contain mostly one class.

For example, suppose a node contains 12 experiments: 6 normal and 6 overheating. This is a mixed node, so it is hard to make a confident prediction. CART may test $q < 2.8$ L/min. If the left child contains 5 overheating and 1 normal case, while the right child contains 5 normal and 1 overheating case, the split is useful because both children are more class-consistent than the parent. If both children still contain a 50–50 mixture, the split has not helped.

Let p_{mk} be the fraction of observations in region R_m that belong to class k. A pure node has one class probability equal to 1 and all others equal to 0. A mixed node has several nonzero class probabilities. CART can measure this mixing using either **Information Entropy** or **Gini Impurity**:

Definition: Information Entropy & Gini Impurity

$$H(R_m) = -\sum_{k=1}^{K} p_{mk} \log_2(p_{mk}) \tag{5.4}$$

$$G(R_m) = \sum_{k=1}^{K} p_{mk}(1 - p_{mk}) = 1 - \sum_{k=1}^{K} p_{mk}^2 \tag{5.5}$$

If a region contains only one physical class, for example all observations are normal operation, then $p_{m1} = 1$ and both H and G evaluate to zero. Zero impurity means no class mixing remains. During training, CART chooses the split that most reduces this impurity, so that the resulting leaves become increasingly easy to label.

5.4 Hyperparameters and Overfitting

The most important hyperparameter of a single decision tree is usually its **maximum depth**. This is the allowed number of sequential threshold questions from the root to a leaf. A depth of one means the tree asks only one question. A depth of three means the tree may ask up to three questions along a prediction path.

- **Very shallow tree:** easy to interpret, but may underfit because it cannot represent enough regimes.
- **Moderate depth:** often useful in engineering because the top few splits remain explainable and capture the dominant physical hierarchy.
- **Unlimited depth:** dangerous with small or noisy data. The tree can memorize individual experiments, sensor errors, and outliers.

Depth is therefore a form of regularization. It plays a role similar to polynomial degree in regression: increasing depth increases flexibility, but also increases the risk that the model learns noise instead of physics. Other useful single-tree controls are the minimum number of samples allowed in a leaf, the minimum number of samples required before a node may split, and the choice of splitting criterion. These controls belong to the single decision tree; ensemble models are introduced separately after the single-tree example below.

A First Single-Tree Example

Before combining many trees, consider one small tree. Suppose the goal is to predict the thermal state of a motor-pump module from coolant flow q and heat load P. The tree first asks the most important physical question: is coolant flow low? If the answer is no, the module is usually in normal cooling. If the answer is yes, the tree asks a second question about heat load.

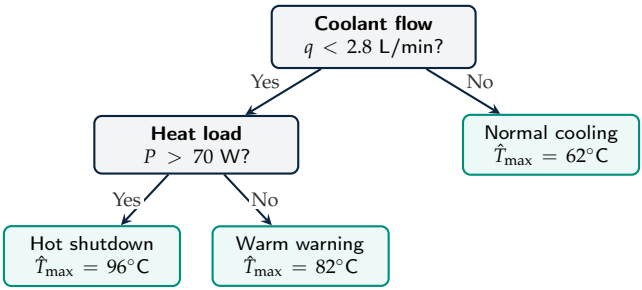


Figure 5.2: **A simple single decision tree.** The root question separates low-flow experiments from normal cooling. Only the low-flow branch needs a second question about heat load. Each leaf stores a learned numerical prediction for maximum temperature.

This example should be read as a learned sequence of if–then tests: if $q \geq 2.8$, predict normal cooling; if $q < 2.8$ and $P > 70$, predict hot shutdown; if $q < 2.8$ and $P \leq 70$, predict a warm warning. Ensembles extend this idea by combining many such trees.

5.5 Ensemble: Bagging, Boosting & Stacking

Ensemble models are introduced here because they address a central limitation of individual decision trees: high variance. **Random Forests** are robust nonlinear models for tabular data, substantially more stable than single deep trees, and often provide a strong reference model before more complex deep-learning architectures are considered.

A single decision tree grown very deep can perfectly memorize the training data. Referring back to the Bias-Variance Decomposition (Lesson 1), a deep tree may achieve near-zero bias but suffer from catastrophically high variance. To build robust tree-based systems, we combine many trees

into an **ensemble**.

5.5.1 Variance Reduction: Bagging & Random Forests

Bootstrap Aggregating (Bagging) is a general ensemble method. To build one bootstrap dataset, we sample rows from the original dataset $\mathcal{D}$ until we have a new dataset of the same size. After a row is selected, it is placed back into the pool before the next draw. Therefore, the same experiment may appear more than once in a bootstrap dataset, while another experiment may be absent. This procedure is called **sampling with replacement**. We repeat it B times and train a separate tree $\hat{f}_b(x)$ on each bootstrap dataset. For regression, the ensemble prediction is their average:

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B}\hat{f}_b(x) \tag{5.6}$$

The benefit of averaging depends on whether the trees make *different* errors. If all trees make almost the same mistake on the same test point, averaging them does not help much. If their errors are partly independent, the mistakes cancel each other.

Let σ^2 denote the variance of one tree's prediction error. Let ρ denote the average correlation between the errors of two different trees. This is the meaning of **pairwise correlation**: choose two trees in the ensemble and ask how similarly they tend to be wrong. If $\rho \approx 1$, the trees are almost identical in their mistakes. If $\rho \approx 0$, their mistakes are much less synchronized.

Under this simplified model, the variance of the averaged prediction is:

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2 \tag{5.7}$$

The second term decreases as we add more trees. The first term does not disappear, because it represents shared error. For example, if every tree repeatedly uses coolant flow as the first split and therefore makes similar mistakes in the same operating regime, those mistakes are correlated and cannot be averaged away.

Self-study connection: In the accompanying homework, Part A asks about the limit $B \to \infty$. The useful hint is to inspect which term in Equation 5.7 contains B and which term does not. Part B asks why Random Forests improve on ordinary Bagging; the hint is to focus on the node-splitting

step, not on the final averaging step. If every tree is allowed to choose from all p features at every split, the same dominant sensors may appear in many trees and keep ρ large.

Random Forests reduce this correlation. At each node split, the algorithm is only allowed to choose from a random subset of features, often $m \approx \sqrt{p}$ for classification. This prevents the same strongest feature from dominating every tree and forces different trees to explore different plausible explanations. The final forest is usually less interpretable than one tree, but it is much more stable.

5.5.2 Bias Reduction: Gradient Boosting

While Bagging and Random Forests are **parallel variance-reduction** methods, **Boosting** is a **sequential bias-reduction** method. Rather than building independent trees in parallel and averaging them, Gradient Boosting builds weak estimators (shallow trees) *sequentially*. Each new tree $h_m(x)$ is forced to predict the residual errors of the previous ensemble:

$$F_M(x) = F_0(x) + \nu \sum_{m=1}^{M} \gamma_m h_m(x) \tag{5.8}$$

where $\nu \in (0, 1]$ is the learning rate (shrinkage) and γ_m is the step size. Boosting acts as gradient descent in function space, transforming weak, high-bias models into a highly precise additive model.

Modern boosted-tree implementations such as **XGBoost**, **LightGBM**, and **CatBoost** are widely used in engineering and industrial tabular problems. They add efficient optimization, regularization, missing-value handling, and careful control of tree complexity. The key conceptual idea remains the same: many simple trees are added sequentially so each new tree corrects the previous model's mistakes.

5.5.3 Model Synthesis: Stacking

Stacked Generalization (Stacking) combines entirely different algorithmic families. Let $f_1, \ldots, f_K$ be K diverse base models (e.g., an SVM, a Random Forest, and a Gaussian Process). We train a *Meta-Learner* f_{meta} to output the final prediction using the outputs of the base models as its features:

$$\hat{y} = f_{\text{meta}}\Big(f_1(x), f_2(x), \ldots, f_K(x)\Big) \tag{5.9}$$

Stacking learns exactly when to trust the linear robustness of an SVM versus the discontinuous splitting of a Random Forest.

5.6 Comparing the Ensemble Methods

To select the correct ensemble architecture for an engineering problem, we must understand their mathematical tradeoffs:

- **Bagging (Random Forests):** *Parallel execution.* Primary goal: reduce **variance**. Best used when deep trees are useful but unstable. Random Forests are commonly used as strong reference models for tabular engineering data.
- **Boosting (AdaBoost, XGBoost):** *Sequential execution.* Primary goal: reduce **bias**. Best used when shallow trees underfit but can be combined into a precise additive model. Highly accurate, but more sensitive to learning rate, number of trees, and depth.
- **Stacking:** *Hierarchical execution.* Primary goal: **Synthesize diverse representations.** Best used when the physical system has both continuous and discontinuous regions, requiring a meta-learner to merge the strengths of fundamentally different algorithms.

5.7 Case Study: Cooling a Motor-Pump Module

Case Study: Thermal Safety and Performance Prediction

The Setup: An engineering team tests a compact motor-pump module used to circulate coolant. The measured features are coolant flow rate q, electrical heat load P, ambient temperature T_{amb}, and vibration level v. The team wants two predictions: the maximum casing temperature T_{max} and whether the module enters an overheating regime.

Cross-Disciplinary Reading: This example is not specific to chemical engineering. Mechanical engineers see it as thermal management, electrical engineers see it as power dissipation, physicists see heat balance, computer scientists see tabular supervised learning, and chemical engineers see the same structure in pumps, jackets, condensers, and reactors.

The Physical Threshold: When coolant flow is sufficient, temperature rises smoothly with heat load. When coolant flow drops below a critical value, heat removal collapses and the module enters a different regime. This is exactly the kind of threshold behavior a tree can learn.

Modeling Strategy:

- A **single tree** gives a readable diagnostic hierarchy.
- A **shallow tree** may be used as a transparent engineering rule.
- A **deep tree** may overfit sensor noise and rare test conditions.
- A **Random Forest** averages many trees to obtain a much more reliable predictor.

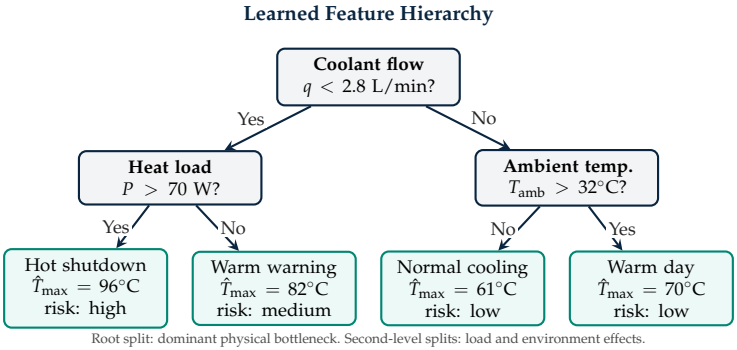


Figure 5.3: **Interpretable decision tree for thermal safety.** The tree predicts both a continuous temperature estimate and a qualitative overheating risk. The first split exposes the most important learned feature: coolant flow.

Figure 5.3 illustrates the interpretability of a trained decision tree. The model identifies coolant flow as the primary split because it is the dominant thermal bottleneck, while heat load and ambient temperature refine the terminal prediction. The leaf values are estimated from training data and therefore constitute learned model outputs.

Sample	Coolant flow q (L/min)	Heat load P (W)	Ambient temp. T_{amb} (°C)	Tree leaf / prediction
1	1.4	85	27	Hot shutdown: $\hat{T}_{max} = 96$°C, high risk
2	2.1	78	31	Hot shutdown: $\hat{T}_{max} = 96$°C, high risk
3	2.6	92	35	Hot shutdown: $\hat{T}_{max} = 96$°C, high risk
4	1.8	62	29	Warm warning: $\hat{T}_{max} = 82$°C, medium risk
5	2.5	68	34	Warm warning: $\hat{T}_{max} = 82$°C, medium risk
6	3.2	58	25	Normal cooling: $\hat{T}_{max} = 61$°C, low risk
7	4.6	82	30	Normal cooling: $\hat{T}_{max} = 61$°C, low risk
8	5.1	70	28	Normal cooling: $\hat{T}_{max} = 61$°C, low risk
9	3.4	66	35	Warm day: $\hat{T}_{max} = 70$°C, low risk
10	4.2	88	36	Warm day: $\hat{T}_{max} = 70$°C, low risk

Table 5.1: Representative observations consistent with the learned tree in Figure 5.3. Each row follows the displayed threshold rules and terminates in the corresponding prediction leaf.

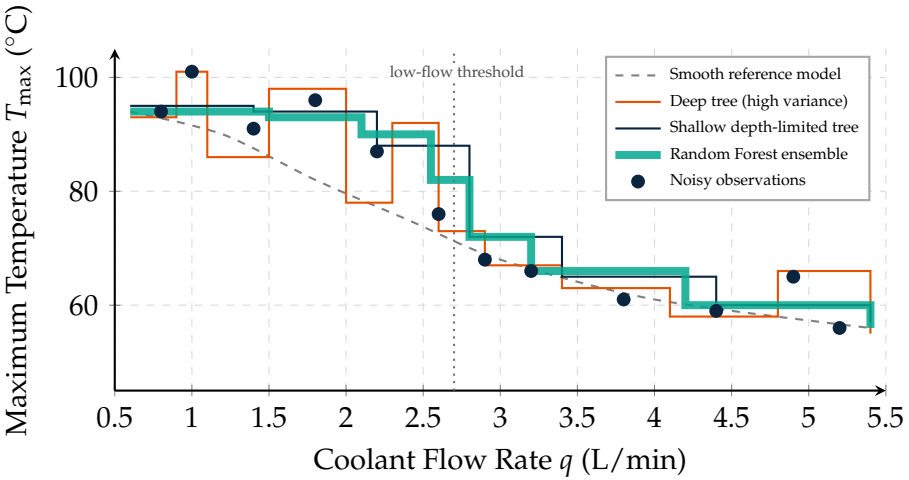


Figure 5.4: **Depth and ensemble behavior in a threshold system.** A deep single tree may fit noisy measurements with high variance, a shallow tree may underfit relevant structure, and a Random Forest stabilizes the learned thermal threshold by averaging many decorrelated trees.

5.8 Summary

- **Decision trees are learned models:** the feature choices, thresholds, and leaf values are fitted from data, not hand-written IF–THEN rules.
- **Trees are useful for both regression and classification:** the same split logic can predict temperatures, yields, failure times, or class

labels.

- **Interpretability is a real advantage:** a tree provides a visual model and a natural feature hierarchy.
- **Hyperparameters matter:** maximum depth controls the tradeoff between readable generalization and memorized noise.
- **Random Forests are powerful:** they turn unstable single trees into robust ensemble predictors for tabular engineering data.
- **Boosted trees are important extensions:** algorithms such as XGBoost build strong models by sequentially correcting residual errors.